BankTech LLM Financial Advisor
Module 2 Project: Security and Compliance Framework

Prepared for Peer Review submission | Generated: 2026-02-12 09:26

Scenario: BankTech is launching an LLM-powered financial advisor for 5 million customers. A penetration test exposed unsecured API endpoints, prompt injection susceptibility, and inadequate audit logging. This project implements a layered security framework to pass regulatory audits.

Learning Objectives Covered

- LO1: Configure OAuth 2.0/JWT authentication and IAM authorization for LLM API endpoints.
- LO2: Implement rate limiting in API Gateway to prevent abuse and cost attacks.
- LO3: Build prompt injection defenses via validation and sanitization.
- LO4: Deploy output filtering to prevent PII leakage.
- LO5: Establish audit logging meeting GDPR, SOC 2, and HIPAA evidence expectations.

Architecture Overview

```
Client Apps (Mobile/Web)
   |
   | 1) OAuth2/JWT (Authorization: Bearer <token>)
   v
API Gateway (Kong)
 - JWT validation (issuer + expiration)
 - Consumer-based rate limiting (Basic/Pro/Enterprise)
 - Security headers + request size limits
 - Forwarded correlation headers (X-Request-Id, X-Consumer-Id)
   |
   v
LLM Middleware (Python)
 - Input validation (prompt injection detection)
 - Prompt sanitization (length caps, escape stripping)
 - Output filtering (PII detection + redaction/block)
 - Security event logging
   |
   v
Inference Service / Model Router
   |
   v
Audit Logging Pipeline (ELK)
 - Structured JSON logs (prompt hash, token counts, durations)
 - RBAC + encryption + long retention
 - GDPR right-to-deletion workflow (pseudonymization)
```

Key design choices

• Defense-in-depth: gateway controls stop most abuse; middleware stops LLM-specific attacks; logging provides forensics and compliance evidence.

• Privacy-first logging: never store full prompts or raw PII. Use hashes, masked values, and least-privilege access.

• Auditor-friendly evidence: deterministic test cases, HTTP status evidence (401/429/403), and a compliance matrix mapping controls to frameworks.

Requirement 1: API Authentication Configuration (OAuth2/JWT) + IAM Authorization

Deliverables: API Gateway configuration with JWT authentication; IAM policy documents with least-privilege permissions; evidence of rejected unauthorized requests (401); token validation test results.

1.1 Kong JWT plugin configuration

```
- name: jwt
    service: llm-inference-service
    config:
      uri_param_names: []
      cookie_names: []
      header_names:
        - Authorization
      claims_to_verify:
        - exp
        - iss
      key_claim_name: iss
      secret_is_base64: false
      run_on_preflight: true
      maximum_expiration: 3600  # 1 hour max token lifetime

  # ------------------------------------------------------------------------
  # SOLUTION:
```

1.2 IAM authorization

```
1.2 IAM policies (least privilege)

Inference server role:
- s3:GetObject on arn:aws:s3:::banktech-model-artifacts/models/*
- logs:CreateLogGroup, logs:CreateLogStream, logs:PutLogEvents on arn:aws:logs:*:*:log-group:/banktech/llm/*

Application caller role:
- execute-api:Invoke only for POST /v1/inference on the specific API stage
- deny wildcard resources and admin privileges
```

1.3 Evidence of rejected requests (401)

```
1.3 Token validation tests (capture screenshots/logs)

# Missing token -> 401
curl -i https://<gateway-host>/v1/inference -d '{"prompt":"hello"}'

# Invalid or expired token -> 401
curl -i -H "Authorization: Bearer <invalid_or_expired>" https://<gateway-host>/v1/inference -d '{"prompt":"hel

Evidence checklist:
- HTTP/1.1 401 Unauthorized
- gateway log entry for JWT verification failure (without storing prompt text)
```

Requirement 2: Rate Limiting Implementation (Tiered Quotas + Burst Control)

Deliverables: rate limiting for three tiers (Basic 1,000/hour; Pro 5,000/hour; Enterprise 20,000/hour); evidence of 429 when exceeded; response headers showing quota status; burst limits for traffic spikes.

2.1 Kong rate-limiting configuration (consumer-based)

```
Rate Limiting - Basic Tier (1,000 requests/hour)
  # ------------------------------------------------------------------------
  - name: rate-limiting
    consumer: basic-tier-client
    config:
      hour: 1000
      minute: 100
      policy: redis
      redis_host: ${REDIS_HOST}
      redis_port: 6379
      redis_password: ${REDIS_PASSWORD}
      redis_database: 0
      fault_tolerant: true
      hide_client_headers: false
      error_code: 429
      error_message: "Rate limit exceeded. Please upgrade your plan or try again later."

  # ------------------------------------------------------------------------
  # SOLUTION: Rate Limiting - Professional Tier (5,000 requests/hour)
  # ------------------------------------------------------------------------
  - name: rate-limiting
    consumer: professional-tier-client
    config:
      hour: 5000
      minute: 500
      policy: redis
      redis_host: ${REDIS_HOST}
      redis_port: 6379
      redis_password: ${REDIS_PASSWORD}
      redis_database: 0
      fault_tolerant: true
      hide_client_headers: false

  # ------------------------------------------------------------------------
  # SOLUTION: Rate Limiting - Enterprise Tier (20,000 requests/hour)
  # ------------------------------------------------------------------------
  - name: rate-limiting
    consumer: enterprise-tier-client
    config:
      hour: 20000
      minute: 2000
      policy: redis
      redis_host: ${REDIS_HOST}
      redis_port: 6379
      redis_password: ${REDIS_PASSWORD}
      redis_database: 0
      fault_tolerant: true
      hide_client_headers: false

  # ------------------------------------------------------------------------
  # CORS - Cross-origin resource sharing
  # ------------------------------------------------------------------------
  - name: cors
    service: llm-inference-service
    config:
```

```
      origins:
        - "https://app.banktech.com"
        - "https://staging.banktech.com"
        - "https://admin.banktech.com"
      methods:
        - GET
        - POST
        - OPTIONS
      headers:
        - Authorization
        - Content-Type
        - X-Request-ID
        - X-Correlation-ID
      exposed_headers:
        - X-RateLimit-Remaining
        - X-RateLimit-Limit
        - X-RateLimit-Reset
      credentials: true
      max_age: 3600

  # -----------------------------------------------------------------------------
  #
```

2.2 Evidence of 429 + quota headers

```
2.2 Rate limit verification (capture screenshots/logs)

# Generate enough requests to exceed the tier:
for i in {1..120}; do
  curl -s -o /dev/null -w "%{http_code}\n" -H "Authorization: Bearer <valid>" https://<gateway-host>/v1/infere
done

Expected:
- HTTP 429 Too Many Requests
- Headers like:
  X-RateLimit-Limit-Hour, X-RateLimit-Remaining-Hour, Retry-After
```

Requirement 3: Prompt Injection Defense (Input Validation + Sanitization)

Deliverables: input validation middleware code; sanitization logic (regex + normalization + length limit); test results showing blocked injection attempts; security event logging configuration.

3.1 Defense-in-depth design

```
3.1 Detection layers implemented

Layer A: Regex patterns (OWASP LLM01 prompt injection)
- instruction override: ignore previous, override system, [system]/[admin]
- jailbreak: DAN, developer mode, pretend no restrictions, roleplay
- prompt leaking: reveal system prompt, print initial instructions verbatim
- data exfiltration: export all data, list all customers/accounts
- SQL/NoSQL: UNION SELECT, DROP TABLE, {"$gt":""}
- encoded: base64/hex/url encoding, "decode and execute"

Layer B: Keyword + structure scoring
- high-risk terms (password, api_key, admin, sudo, etc.)
- delimiter overuse and nested instruction markers

Sanitization
- strip control chars and escape sequences
- normalize whitespace
- enforce max prompt size (4096 tokens policy; hard cap on characters)
```

3.2 Injection test suite coverage

Category	Cases	Severity
direct_injection	5	CRITICAL
indirect_injection	3	HIGH
pii_extraction	4	CRITICAL
sql_nosql_injection	2	CRITICAL
legitimate_requests	4	INFO

3.3 Test evidence to include

```
3.3 Test results (evidence plan)

Run the provided injection_tests.json cases through the middleware:
- expected BLOCKED -> HTTP 403 and SECURITY_ALERT log event
- expected ALLOWED -> forwarded to inference

Capture:
- blocked examples from DI-001..DI-005, II-001..II-003, PII-001..PII-004, SQL-001..SQL-002
- allowed examples from LEG-001..LEG-004
```

Requirement 4: PII Output Filtering (Leakage Prevention)

Deliverables: output filtering config for PII patterns; financial redaction rules; test results showing PII detection and blocking; redaction examples.

4.1 Rules and actions

```
4.1 PII and financial redaction rules

Detect and mask:
- SSN -> XXX-XX-XXXX
- Credit cards -> XXXX-XXXX-XXXX-XXXX
- Emails / phones -> [REDACTED]
- Account identifiers (account numbers, IBAN) -> [ACCOUNT REDACTED]
- JWT/API keys -> [TOKEN REDACTED]

Actions:
- Default: redact in response
- If high-risk + policy requires: block response and return safe message
- Always: emit audit log with pii_detected and pii_types (hashed values only)
```

4.2 Examples

```
4.2 Redaction examples

"SSN 123-45-6789" -> "SSN XXX-XX-XXXX"
"Card 4111 1111 1111 1111" -> "Card XXXX-XXXX-XXXX-XXXX"
"Your account GB29NWBK60161331926819" -> "Your account [ACCOUNT REDACTED]"
```

Requirement 5: Compliance-Ready Audit Logging (SOC 2, GDPR, HIPAA)

Deliverables: log schema and ELK configuration; compliance matrix mapping to SOC 2/GDPR/HIPAA; sample audit queries; business analysis (3-5 sentences).

5.1 Log schema

```
5.1 Structured log fields (auditor-friendly)
- timestamp (UTC), event_id, event_type, severity
- request_id/correlation_id/session_id
- user_id, consumer_id, client_ip (masked), user_agent
- endpoint, method, status_code, response_time_ms
- prompt_hash (never raw prompt), token_count_input/output
- pii_detected + pii_types, security_flags + threat_level
- retention_days + compliance frameworks
- event_hash + previous_hash (tamper-evident chain)
```

5.2 ELK and retention

```
5.2 ELK deployment + retention + controls
- Deploy Elasticsearch/Logstash/Kibana (Docker Compose)
- Logstash parses JSON logs and indexes into audit-* indices
- Apply RBAC: only Compliance/Security roles can view audit indices
- Encrypt in transit (TLS) and at rest (disk/KMS)
- Retention via ILM:
  * HIPAA: 7 years
  * SOC 2: 6 years
```

5.3 Compliance matrix excerpt

mentation Status	Implementation Details	Evidence Required	Owner	Review Frequency	Last Review Date	Ne
D	Implement OAuth 2.0 / JWT authentication	Authentication logs showing successful/failed attempts	Security Team	Quarterly	2024-01-01	20
D	Configure consumer tiers in API Gateway	RBAC policy documentation and access logs	Security Team	Quarterly	2024-01-01	20
D	Configure rate limiting plugins	Rate limit configuration and violation logs	Platform Team	Monthly	2024-01-01	20
D	Implement PII filtering on outputs	Log samples showing masked PII	Compliance Team	Monthly	2024-01-01	20
D	Configure SSL/TLS on load balancer	SSL certificate and configuration documentation	Platform Team	Annually	2024-01-01	20
D	Implement consent management	Consent records and privacy policy	Legal/Compliance	Annually	2024-01-01	20
D	Implement data deletion API	Deletion request logs and confirmations	Engineering Team	Quarterly	2024-01-01	20
mented	Prompts not stored only hashed for audit	Architecture documentation	Security Team	Annually	2024-01-01	20
D	Implement comprehensive audit logging	Sample audit logs	Platform Team	Monthly	2024-01-01	20
D	Configure security alerting	Alert configuration and incident logs	Security Team	Monthly	2024-01-01	20
D	Configure log retention policy	Log retention configuration	Platform Team	Annually	2024-01-01	20
D	Implement input validation	Test results from injection_tests.json	Security Team	Monthly	2024-01-01	20

5.4 Auditor queries

```
5.4 Sample audit queries (Kibana)
- Unauthorized requests: event_type:"authentication" AND outcome:"failure"
- Rate limit exceeded: event_type:"rate_limit" AND outcome:"exceeded"
- Blocked injections: event_type:"security_alert" AND outcome:"blocked"
```

```
- PII detections: pii_detected:true
- Long latency outliers: event_type:"api_response" AND response_time_ms:>1000
```

5.5 Business analysis

5.5 Business analysis (3-5 sentences)

The critical vulnerabilities identified in the penetration test are addressed through layered controls: JWT authentication blocks anonymous access, rate limiting reduces abuse and cost attacks, and middleware validation blocks prompt injections before they reach the model. Output filtering prevents PII leakage by redacting sensitive patterns and recording detections in an audit trail without storing raw values. The audit logging schema is suitable for a SOC 2 audit because it captures authentication outcomes, security events, and request/response metadata with tamper-evident integrity and long retention. Before production, additional hardening should include automated ZAP regression scans in CI, secrets management via Vault/KMS, and periodic access reviews for audit index RBAC.

Appendix: Key Artifacts Included

Artifact	Filename
API Gateway config (template)	M2_api_gateway_config.yaml
API Gateway config (solution)	M2_api_gateway_config.solution.yaml
Prompt validator (template)	M2_prompt_validator.py
Prompt validator (solution)	M2_prompt_validator.solution.py
Audit logger (template)	M2_audit_logger.py
Audit logger (solution)	M2_audit_logger.solution.py
Injection test suite	M2_injection_tests.json
Compliance matrix	M2_compliance_matrix.csv

Submission note: attach this PDF to Peer Review. Optionally include screenshots of (a) 401 unauthorized, (b) 429 rate limit, (c) blocked injection tests, (d) PII redaction, and (e) Kibana queries/dashboards for audit evidence.